Hafta 6 — Derinlemesine

> ## ⚠️ BU DOSYA SINAV KAPSAMI DIŞINDADIR
>
> Buradaki hiçbir konudan sınavda soru sorulmaz. Bu dosya, *"peki bu nasıl çalışıyor?"* diye soran öğrenci içindir. Sınav kapsamı `H06-ders-notu.md` dosyasıdır.
>
> Dönem projesinde işinize yarayacak birkaç şey de buradadır (bölüm 2, bölüm 3, bölüm 6).
>
> Bu dosyadaki örneklerin hepsi `Hafta-06/Bitmis` projesinin bir **kopyasında** denendi; "ölçüldü" yazan satırlar o denemenin çıktısıdır. Projenin kendisi değiştirilmedi.

---

1. Doğrulamanın İçeriden İşleyişi

Ders notu bölüm 1.2'de "Model Binding bitince çatı doğrulamayı çalıştırır" dedik. Bu işi yapan sınıfın adı `ValidationVisitor`'dır: modeli bir **ağaç** gibi gezer, her özellik bir düğümdür. Karmaşık bir düğümde önce **çocuklar** (iç özellikler) gezilir, sonra düğümün **kendi** kuralları çalışır. Nesnenin `Validate` metodu da bu "kendi kurallar" arasındadır — çocuklardan biri hatalıysa atlanması (ders notu bölüm 5.3) bu sıradan gelir.

Her düğümün sonucu `ModelState`'te `ValidationState` olarak durur:

Değer	Anlamı
`Unvalidated`	Henüz bakılmadı
`Valid`	Bakıldı, hata yok
`Invalid`	En az bir hata var
`Skipped`	Bilerek atlandı (`[ValidateNever]` ya da sınır aşıldı)

`/Dogrula` sayfasındaki tablonun **Durum** sütunu tam olarak bu değerdir.

En fazla 200 hata

Çatı bir istekte en fazla **200** hata toplar; sonra durur. Sınırı değiştirmek için:

```
builder.Services.AddControllersWithViews(o =>
{
    o.MaxModelValidationErrors = 50;
});
```

Davranışı görmek için sınırı **3** yapıp `/Dogrula`'yı boş gönderdik (ölçüldü):

Anahtar	Durum	Hata

(boş anahtar)	Invalid	`TooManyModelErrorsException` — mesajsız
`Eposta`	Invalid	The E-posta field is required.
`Yas`	Invalid	The field Yaş must be between 18 and 99.
`Parola`	**Valid**	—
`ParolaTekrar`, `Telefon`, `WebSitesi`, `Boy`	Skipped	—

İki şaşırtıcı satır var: üçüncü hata yerine **boş anahtara** bir istisna yazıldı, ve `Parola` boş olduğu hâlde **Valid** görünüyor — hatası sınıra takılıp yazılmadı. Sınıra yaklaşan formlarda tek tek alan durumlarına güvenmeyin; `IsValid` yine `false`'tur.

`[ValidateNever]` — bir özelliği hiç doğrulamamak

```
using Microsoft.AspNetCore.Mvc.ModelBinding.Validation;

[ValidateNever]
[Required]
public string? Ad { get; set; }
```

Ölçülen: `Ad` boş gönderildi, `[Required]` yazılı olduğu hâlde **Skipped**. Model Binding'i etkilemez; değer yine bağlanır, yalnızca denetlenmez. Hafta 8'de EF Core entity'lerinin gezinme özelliklerinde (`Yayinevi` gibi) bu attribute'le karşılaşacaksınız — Hata Köşesi'ndeki sorunun bir çözüm yolu da budur.

İlgili diğer ayarlar

Ayar	Varsayılan	Ne yapar
`MaxValidationDepth`	32	İç içe nesnelerde en fazla bu kadar derine inilir
`ValidateComplexTypesIfChildValidationFails`	`false`	`true` yapılırsa `Validate`, özellik hataları olsa bile çağrılır (ders notu bölüm 5.3)
`SuppressImplicitRequiredAttributeForNonNullableReferenceTypes`	`false`	`true` yapılırsa örtük `[Required]` kapanır (ders notu bölüm 1.3)

Action parametrelerinin doğrulanması

Doğrulama yalnızca ViewModel'in özelliklerine değil, Learn'e göre **action parametrelerinin kendisine** de uygulanır:

```
public IActionResult Ara([Required, StringLength(50)] string? kelime)
{
    if (!ModelState.IsValid) return View();   // kelime boşsa buraya düşer
    ...
}
```

Elle yeniden doğrulamak

Action içinde modelin bir özelliğini **siz** değiştirdiyseniz, `ModelState` eski sonucu tutmaya devam eder. Yeniden doğrulatmak için önce eski durumu silin:

```
girdi.Baslik = girdi.Baslik.Trim();
ModelState.ClearValidationState(nameof(girdi.Baslik));
if (!TryValidateModel(girdi)) return View(girdi);
```

Kaynak: Model validation in ASP.NET Core MVC

---

2. Kendi Attribute'ünüzü İstemcide Çalıştırmak: `IClientModelValidator`

Ders notu bölüm 5.2'de `[BuyukHarfleBaslar]` yalnızca sunucuda çalışıyordu: HTML'e hiçbir `data-val-*` yazmıyordu, istemci onu bilmiyordu. Onu istemciye taşımak **üç parça** ister.

1 — Attribute HTML'e kendi `data-val-*`'ını yazar:

```
using Microsoft.AspNetCore.Mvc.ModelBinding.Validation;

public class BuyukHarfleBaslarAttribute : ValidationAttribute, IClientModelValidator
{
    // ... IsValid aynı kalır ...

    public void AddValidation(ClientModelValidationContext context)
    {
        context.Attributes.TryAdd("data-val", "true");
        context.Attributes.TryAdd("data-val-buyukharf",
            FormatErrorMessage(context.ModelMetadata.GetDisplayName()));
    }
}
```

2 — Aynı kuralı JavaScript'te yazarsınız (`wwwroot/js/buyuk-harf.js`):

```
(function ($) {
    if (!$ || !$.validator) return;

    $.validator.addMethod('buyukharf', function (value, element) {
        if (this.optional(element)) return true;
        var ilk = value.charAt(0);
        return ilk !== ilk.toLocaleLowerCase('tr-TR');
    });

    $.validator.unobtrusive.adapters.addBool('buyukharf');
})(window.jQuery);
```

`addMethod` kuralı jQuery Validation'a tanıtır; `addBool` unobtrusive'e "`data-val-buyukharf` gördüğünde bu kuralı bağla" der.

3 — Betik, unobtrusive'ten sonra yüklenir (`_ValidationScriptsPartial.cshtml` 'in sonuna):

```
<script src="~/js/buyuk-harf.js" asp-append-version="true"></script>
```

Ölçülen (`/Kitap/EkleTH` , istemci doğrulaması açık):

Başlık	Sonuç
Ctrl+U	`data-val-buyukharf="Başlık büyük harfle başlamalıdır."` artık HTML'de
`küçük harfli`	İstek **gitmedi**; mesaj anında
`İstanbul Hatırası`	Geçti (Türkçe `İ` doğru tanındı)
`1984`	İstek **gitmedi**; rakam büyük harf sayılmıyor

Son satır bir karar ister: sunucudaki `char.IsUpper('1')` de `false` döner, yani iki taraf **tutarlı** — ama "1984" adlı bir kitabı kabul etmek istiyorsanız kuralı **iki yerde** değiştirmeniz gerekir. Bu yöntemin bedeli budur: aynı kural iki dilde, iki kez yazılır.

> 💡 Aynı işin ikinci bir yolu da var: attribute'e dokunmadan ayrı bir `AttributeAdapterBase<T>` sınıfı yazıp `IValidationAttributeAdapterProvider` ile kaydetmek. Attribute başka bir projede (sınıf kütüphanesinde) duruyorsa bu yol seçilir.

Kaynak: Custom client-side validation

---

3. Bağlama Mesajlarını Türkçeleştirmek: `ModelBindingMessageProvider`

Hafta 5 detay bölüm 6'da doğrulama **attribute'lerinin** mesajlarını Türkçeleştirmenin yollarını gördük. Ama bu hafta iki kaynaktan mesaj geldiğini öğrendik (ders notu bölüm 1.2): attribute'ler ve **bağlama**. `The value '' is invalid.` bir attribute'ten değil, bağlamadan gelir; `ErrorMessage` onu değiştiremez.

Bağlama mesajları `MvcOptions` üzerinden değiştirilir:

```
builder.Services.AddControllersWithViews(o =>
{
    var m = o.ModelBindingMessageProvider;
    m.SetValueMustNotBeNullAccessor(_ => "Bu alan boş bırakılamaz.");
    m.SetAttemptedValueIsInvalidAccessor((deger, alan) => $"'{deger}', {alan} için geçerli bir değ
    m.SetValueMustBeANumberAccessor(alan => $"{alan} bir sayı olmalıdır.");
});
```

Ölçülen (`/Dogrula` , istemci doğrulaması kapalı):

Gönderilen	Önce	Sonra
`Yas` boş (`int`)	The value '' is invalid.	**Bu alan boş bırakılamaz.**
`Boy` = `abc` (`decimal?`)	The value 'abc' is not valid for Boy (m).	**'abc', Boy (m) için geçerli bir değer değil.**
`Boy` alanının HTML'i	`data-val-number="The field Boy (m) must be a number."`	`data-val-number="Boy (m) bir sayı olmalıdır."`
`Ad` boş	The Ad field is required.	The Ad field is required. — **değişmedi**

Son satır iki kaynağın ayrı olduğunu gösteriyor: `Ad` 'ın mesajı `[Required]` attribute'ünden gelir, bağlama sağlayıcısı ona dokunmaz. Üçüncü satır da ilginç: `ValueMustBeANumber` sunucuda değil, **istemcinin** mesajında kullanılıyor.

> ⚠️ `SetValueMustNotBeNullAccessor` 'a gelen parametre alanın adı değil, **gönderilen değerdir** (burada boş metin). Mesajda alan adı istiyorsanız tipi nullable yapıp `[Required]` yazmak daha temiz bir yoldur (ders notu bölüm 2.3).

---

4. `[Remote]` : İstemciden Sunucuya Soran Doğrulama

"Bu başlıkta kitap var mı?" sorusu tarayıcıda cevaplanamaz; liste sunucuda. `[Remote]` istemciye "bu alan değişince şu adrese sor" der:

```
// ViewModel
[Remote("BaslikUygun", "Kitap", ErrorMessage = "Bu başlıkta bir kitap zaten var.")]
public string Baslik { get; set; } = "";

// KitapController
[AcceptVerbs("GET", "POST")]
public IActionResult BaslikUygun(string baslik) =>
    Json(!KitapDeposu.Hepsi.Any(k => string.Equals(k.Baslik, baslik?.Trim(),
        StringComparison.CurrentCultureIgnoreCase)));
```

Action `true` dönerse alan geçerli, `false` dönerse attribute'teki mesaj gösterilir.

Ölçülen:

Deneme	Sonuç
Ctrl+U	`data-val-remote-url="/Kitap/BaslikUygun"`
`Suç ve Ceza` yazıp alandan çıkmak	Network'te `GET /Kitap/BaslikUygun?Baslik=Su%C3%A7%20ve%20Ceza` ; mesaj alanın altında
Aynısını konsoldan `document.forms[0].submit()` ile göndermek (TODO 6.4 **kaldırılmış** kopyada)	**302** — aynı başlıklı ikinci kitap **kaydedildi**

Son satır bölümün asıl dersi: `[Remote]` **yalnızca istemcide** çalışır, sunucudaki doğrulamada hiçbir şey denetlemez. İstemci doğrulamasının bir parçasıdır, dolayısıyla ders notu bölüm 4.3'teki atlatma yollarının hepsi ona da işler. Sunucudaki `AddModelError` (TODO 6.4) yerinde kalmalı; `[Remote]` yalnızca kullanıcıya erken haber verir.

Kaynak: [Remote] attribute

---

5. FluentValidation: Kuralları Modelin Dışına Taşımak

DataAnnotations kuralı modelin **üstüne** yazar. Bazı ekipler kuralların ayrı bir sınıfta durmasını tercih eder; bunun için yaygın kütüphane **FluentValidation**'dır (çatının parçası değil, NuGet paketi).

```
using FluentValidation;

public class KitapEkleValidator : AbstractValidator<KitapEkleVM>
{
    public KitapEkleValidator()
    {
        RuleFor(k => k.Baslik).NotEmpty().Length(2, 100);
        RuleFor(k => k.Fiyat).NotNull().InclusiveBetween(1, 10000);
        RuleFor(k => k.Yazar)
            .NotEqual(k => k.Baslik, StringComparer.CurrentCultureIgnoreCase)
            .WithMessage("Başlık ile yazar aynı olamaz.");
    }
}
```

Son kural, ders notunda `IValidatableObject` ile yazdığımız kuralın karşılığıdır.

Belgelerinin bugünkü önerisi doğrulayıcıyı action'da **elle** çağırmaktır; çatının doğrulama hattına otomatik bağlanan eski paket (`FluentValidation.AspNetCore`) için "*artık yeni projelerde önermiyoruz*" yazıyor:

```
// Program.cs  (paket: FluentValidation.DependencyInjectionExtensions)
builder.Services.AddValidatorsFromAssemblyContaining<KitapEkleValidator>();

// KitapController
public async Task<IActionResult> EkleTH(KitapEkleVM girdi,
    [FromServices] IValidator<KitapEkleVM> dogrulayici)
{
    var sonuc = await dogrulayici.ValidateAsync(girdi);
    foreach (var hata in sonuc.Errors)
        ModelState.AddModelError(hata.PropertyName, hata.ErrorMessage);

    if (!ModelState.IsValid) return View(girdi);
    ...
}
```

Hatalar yine `ModelState`'e yazıldığı için `asp-validation-for` ve kanonik kalıp değişmeden çalışır. Bedeli: kurallar artık `data-val-*` olarak **istemciye taşınmaz**.

> Bu bölümdeki kod derste denenmedi; paketin belgelerinden alındı. Kaynak: FluentValidation — ASP.NET Core

---

6. Kültür ve Sayılar

`UseRequestLocalization`'ın içi

Ders notu bölüm 4.4'te kültürü `tr-TR`'ye sabitledik. Middleware her istekte kültürü üç kaynaktan sırayla arar: query string (`?culture=`), bir cookie, tarayıcının `Accept-Language` başlığı. Bulduğu kültür `SupportedCultures`'ta **yoksa** varsayılana döner.

Ölçülen: `?culture=en-US` ve `Accept-Language: en-US` ile istenen sayfa yine `98,40 TL` yazdı. Listede yalnızca `tr-TR` olduğu için isteğin ne dediği önemli değil — sabitlemenin amacı da buydu.

Middleware'in **yeri** önemlidir: kültürü kullanan her şeyden (routing, controller'lar) **önce** çalışmalı. Bizim projede `var app = builder.Build();` satırının hemen altında duruyor.

Hangi kültür hangi sayıyı bağlar

Hafta 4 detay bölüm 5'teki tablonun ölçülmüş hâli. Aynı proje üç kültürle çalıştırıldı, `Fiyat` alanına üç değer konsoldan gönderildi:

Sunucu kültürü	`12,50`	`12.50`	`12.5`
`tr-TR`	✅ 12,50	⚠️ **1250**	⚠️ **125**
`en-US`	⚠️ **1250**	✅ 12.50	✅ 12.5
`InvariantCulture`	⚠️ **1250**	✅ 12.50	✅ 12.5

Hiçbir hücrede "bağlanmadı" yok: **yanlış ayraç binlik ayracı sayılıyor** ve sayı sessizce büyüyor. Çatının sayı bağlayıcısı binlik ayracına izin verir ve grup uzunluğunu (üç hane) denetlemez. Hata mesajı olmadığı için bu, ders notunun en tehlikeli satırlarından biridir.

`type="number"` bir çözüm mü?

Hafta 5'te `decimal` alanların neden `type="text"` üretildiğini gördük. Elle `type="number"` yazarsanız iki şey olur (ikisi de ölçüldü):

1. **Tarayıcı değeri her zaman noktalı gönderir.** Kullanıcı ekranda virgül görse bile gövdede `12.5` gider. Sunucu `tr-TR` ise bu **125** olarak bağlanır.
2. **Sunucunun yazdığı değeri tarayıcı okuyamaz.** `tr-TR` sunucu `value="120,50"` yazar; `type="number"` bunu geçersiz sayıp alanı **boş** gösterir (ölçülen: `.value` → `""`). Düzenleme formunda kullanıcı mevcut fiyatı hiç görmez.

Yani `type="number"`, sunucu **noktalı** bir kültürle (`en-US` ya da invariant) bağlıyorsa çalışır; `tr-TR` ile birlikte kullanılamaz. Bu projede seçilen yol tersidir: `type="text"`, sunucu `tr-TR`, istemcide `dogrulama-tr.js`.

---

7. Tarayıcının Yerleşik Denetimi ve `novalidate`

Ders notu bölüm 1.4'te üç katmandan söz ettik: tarayıcının kendi denetimi (`type="email"`, `maxlength`), jQuery Validation ve sunucu. İlk ikisi aynı anda çalışmaz:

Sayfa	`<form>`'un `novalidate` attribute'ü
Başlangıç, `/Kitap/EkleTH` (betik yok)	yok — tarayıcı kendisi denetler
Bitmiş, `/Kitap/EkleTH` (betikler yüklü)	`novalidate` — jQuery Validation ekledi (ölçüldü)

jQuery Validation sayfa açılırken forma `novalidate` ekler ve denetimi devralır; tarayıcının balonları yerine bizim mesajlarımız görünür. Betikler yüklenmezse tarayıcı yine kendi denetimini yapar — ders notundaki "sayfada jQuery yokken bile tarayıcı `xyz`'yi göndermedi" ölçümü buradan gelir.

Tek bir düğmenin bütün denetimleri atlaması için HTML'in kendi attribute'ü var:

```
<button type="submit" formnovalidate>Taslak olarak kaydet</button>
```

jQuery Validation da bu attribute'ü tanır. Sunucu yine doğrular — "taslak" kaydı gerçekten istiyorsanız action'da ayrıca ele almanız gerekir.

---

8. Daha İleri Gitmek İsteyenlere

Konu	Nereden başlanır
Kendi `IModelValidatorProvider`'ınız	Attribute kullanmadan doğrulama kuralı eklemek

`IValidationMetadataProvider`	Örtük `[Required]` gibi attribute'lerin **mesajını** merkezden değiştirmek
`[BindRequired]`	Alanın gövdede **hiç gelmemesini** yakalamak (Hata Köşesi'nin tersi)
Minimal API'lerde doğrulama	.NET 10'da `AddValidation()`; Hafta 12
`ProblemDetails` ve `ValidationProblemDetails`	API'lerde doğrulama hatasının biçimi; Hafta 12

Kaynak: Model validation in ASP.NET Core MVC